

A significant part of these lecture notes is developed in [CDG⁺97]. There was already a survey of these techniques at the CCL conference in 1994 [Dau94].

2.8 Presburger Arithmetic and Classical Word Automata

We recall briefly the standard definitions in sections 2.8.1 and 2.8.2. Then we show how automata recognize sets of integers and we give a construction for the decision of arithmetic constraints.

2.8.1 Presburger Arithmetic

The *basic terms* in Presburger arithmetic consist of first-order variables, which we write x, x_1, y, z' ..., the constants 0 and 1 and sums of basic terms. For instance $x + x + 1 + 1 + 1$ is a basic term, which we also write $2x + 3$.

The *atomic formulas* are equalities and inequalities between basic terms. For instance $x + 2y = 3z + 1$ is an atomic formula.

The *formulas* of the logic are first-order formulas built on the atomic formulas. We use the connectives $\wedge$ (conjunction), $\vee$ (disjunction), $\neg$ (negation), $\exists x$ (existential quantification), $\forall x$ (universal quantification). For instance, $\forall x, \exists y. (x = 2y \vee x = 2y + 1)$ is a formula. $\top$ is the empty conjunction (valid formula) and $\perp$ is the empty disjunction (unsatisfiable formula).

The *free variables* of a formula ϕ are defined as usual: for instance $FV(\phi_1 \vee \phi_2) = FV(\phi_1) \cup FV(\phi_2)$ and $FV(\exists x. \phi) = FV(\phi) \setminus \{x\}$.

The interpretation domain of formulas is the set of natural numbers $\mathbb{N}$, in which $0, 1, +, =, \leq$ have their usual meaning. A *solution* of a formula $\phi(x_1, \dots, x_n)$ is an assignment of $x_1, \dots, x_n$ in $\mathbb{N}$ which satisfies the formula. For instance $\{x \mapsto 0; y \mapsto 2; z \mapsto 1\}$ is a solution of $x + 2y = 3z + 1$ and every assignment $\{x \mapsto n\}$ is a solution of $\exists y. (x = 2y \vee x = 2y + 1)$.

2.8.2 Finite Automata

For every word $w \in A^*$, if i is a natural number which is smaller or equal to the length of w , we write $w(i)$ the i th letter of w .

A *finite automaton* on the alphabet A is a tuple (Q, Q_f, q_0, δ) where Q is a finite set of *states*, $Q_f \subseteq Q$ is the set of *final states*, $q_0 \in Q$ is the *initial state* and $\delta \subseteq Q \times Q \times Q$ is the *transition relation*.

Given a word $w \in A^*$ a *run* of the automaton (Q, Q_f, q_0, δ) on w is a word $\rho \in Q^*$ such that $\rho(1) = q_0$ and, if $\rho(i) = q$, $\rho(i+1) = q'$, then $(q, w(i), q') \in \delta$. A *successful run* ρ on w is a run such that $\rho(n+1) \in Q_f$ where n is the length of w . w is *accepted by the automaton* if there is a successful run on w . The *language accepted* (or *recognized*) by an automaton is the set of words on which there is a successful run.

The class of languages which are accepted by some finite automaton is closed by union, intersection and complement. Such constructions can be found in most textbooks. (See e.g. [Per90] for references).

2.8.3 Sets of Integers Recognized by a Finite Automaton

Every natural number can be seen as a word written in base k ($k \geq 1$) over the alphabet $A = \{0, \dots, k-1\}$. For convenience, we write here its representation in a somehow unusual way from right to left. For instance thirteen can be written in base two as: 1011. There are more than one representation for each number, since we may complete the word with any number of 0's on the right: 101100 is another representation of thirteen in base two. Conversely, there is a mapping denoted $\sim^k$ from $\{0, \dots, k-1\}^*$ into $\mathbb{N}$ which associates with each word a natural number: $\widetilde{10110}^2$ is thirteen.

n -uples of natural numbers can be represented in base k as a single word over the alphabet $\{0, \dots, k-1\}^n$ by stacking the representations of each individual numbers. For instance the pair (13, 6) (in base 10) can be represented in base 2 as a word over the alphabet $\{0, 1\}^2$ as: $\begin{smallmatrix} 1011 \\ 0110 \end{smallmatrix}$. Again there are more than one representation since the word can be completed by any number of 0's on the right.

2.8.4 A Translation from Presburger Formulas to Finite Automata

In the logic versus automata spirit we start by associating an automaton with each atomic formula.

Automata associated with equalities. For every basic formula

$$a_1x_1 + \dots + a_nx_n = b \text{ (where } a_1, \dots, a_n, b \in \mathbf{Z})$$

we construct the automaton by saturating the set of rules and the set of states, originally set to $\{q_b\}$ using the inference rule:

$$\frac{q_c \in Q}{q_d \in Q, \quad (q_c, \theta, q_d) \in \delta} \quad \text{if} \quad \begin{cases} a_1\theta_1 + \dots + a_n\theta_n =_2 c \\ d = \frac{c - a_1\theta_1 - \dots - a_n\theta_n}{2} \\ \theta \in \{0, 1\}^n \text{ encodes } (\theta_1, \dots, \theta_n) \end{cases}$$

in other words: for every state $q_c \in Q$, compute the solutions $(\theta_1, \dots, \theta_n)$ of $a_1x_1 + \dots + a_nx_n = c$ modulo 2 and add the state q_d and the rule $q_c \xrightarrow{\theta} q_d$ where $d = \frac{c - a_1\theta_1 - \dots - a_n\theta_n}{2}$.

The initial state is q_b and, if $q_0 \in Q$, then this is the only final state. Otherwise there is no final state.

Example 10. Consider the equation $x + 2y = 3z + 1$.

Since $b = 1$, we have $q_1 \in Q$. We compute the solutions modulo 2 of $x + 2y = 3z + 1$: we get $\{(0, 0, 1), (0, 1, 1), (1, 0, 0), (1, 1, 0)\}$. Then we compute the new states: $\frac{1-0-0+3}{2} = 2$, $\frac{1-0-2+3}{2} = 1$, $\frac{1-1-0+0}{2} = 0$, $\frac{1-1-2+0}{2} = -1$, yielding $q_2, q_0, q_{-1} \in Q$, and the new transitions:

$$q_1 \xrightarrow{\begin{smallmatrix} 0 \\ 0 \\ 1 \end{smallmatrix}} q_2, q_1 \xrightarrow{\begin{smallmatrix} 0 \\ 1 \\ 1 \end{smallmatrix}} q_1, q_1 \xrightarrow{\begin{smallmatrix} 1 \\ 0 \\ 0 \end{smallmatrix}} q_0, q_1 \xrightarrow{\begin{smallmatrix} 1 \\ 1 \\ 0 \end{smallmatrix}} q_{-1}.$$

Going on with states q_2, q_0, q_{-1} , we get the automaton of figure 2.9; we use circles for the states and double circles for final states.

For instance, consider the path from the initial state q_1 to the final state q_0 going successively through $q_{-1}q_{-2}q_0q_1q_2$. The word $\begin{smallmatrix} 111100 \\ 110001 \\ 001110 \end{smallmatrix}$ is accepted by the automaton, which corresponds to the triple (in base ten): $x = 15, y = 35, z = 28$ and one can verify $15 + 2 \times 35 = 3 \times 28 + 1$.

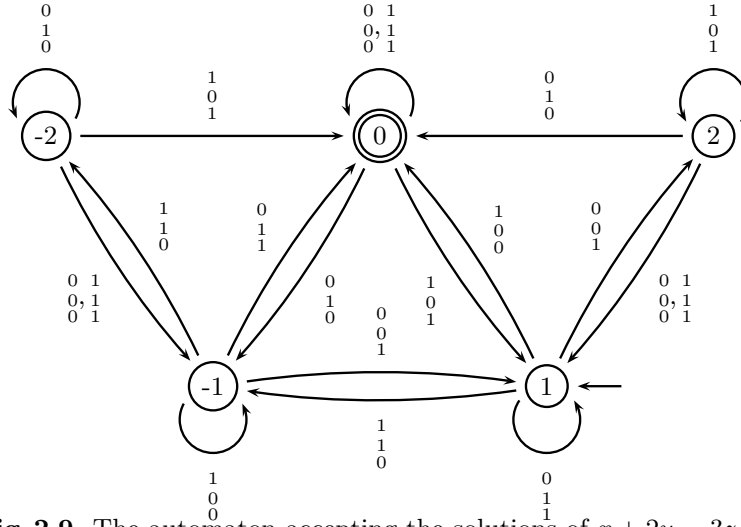


Fig. 2.9. The automaton accepting the solutions of $x + 2y = 3z + 1$

Proposition 8. *The saturation terminates yielding an (deterministic) automaton whose number of states is bounded by $\max(|b|, |a_1| + \dots + |a_n|)$ and which accepts the solutions of $a_1x_1 + \dots + a_nx_n = b$.*

Indeed, it is not difficult to check that the absolute values of the states are bounded by $\max(|b|, |a_1| + \dots + |a_n|)$ (this is an invariant of the inference rule, the proof is left to the reader). The correctness and completeness are also easy: it suffices to show by induction on the length of the word (resp. on the length of the transition sequence) that, any word accepted starting from any state q_c is a solution of $a_1x_1 + \dots + a_nx_n = c$.

The algorithm can be optimized in several respects, for instance pre-computing the solutions modulo 2 or using Binary Decision Diagrams (BDDs) to represent the transitions of the automaton.

Exercise 1. If we fix $a_1, \dots, a_n$, what is the size of the automaton, w.r.t. b ?

Automata associated with inequalities. Next, we compute an automaton for inequalities $a_1x_1 + \dots + a_nx_n \leq b$. The computation is similar: we start with q_b and, from a state q_c we compute the transitions and states as follows. For every bit vector $(\theta_1, \dots, \theta_n)$, $q_c \xrightarrow{\theta_1 \dots \theta_n} q_d$ with

$$d = \lfloor \frac{c - \sum_{i=1}^n a_i \theta_i}{2} \rfloor$$

$$Q_f = \{q_c \mid c \geq 0\}.$$

Example 11. Consider the inequality $2x - y \leq -1$. The automaton which we get using the algorithm is displayed on figure 2.10.

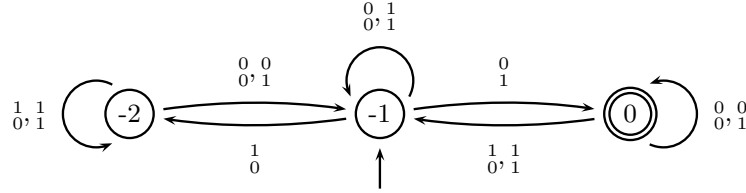


Fig. 2.10. The automaton accepting the solutions of $2x - y \leq -1$

Exercise 2. What is the largest number of reachable states in the automaton for $a_1x_1 + \dots + a_nx_n \leq b$?

Closure properties and automata for arbitrary formulas.

Let $\mathcal{A}_\phi(x_1, \dots, x_n)$ be an automaton over the alphabet $\{0, 1\}^n$ which accepts the solutions of the formula ϕ whose free variables are contained in $\{x_1, \dots, x_n\}$. If a variable x_i does not occur free in ϕ , then accepting/not accepting a word will not depend on what we have on the i th track.

Exercise 3. Show how to compute $\mathcal{A}_\phi(x_1, \dots, x_n, y)$, given $\mathcal{A}_\phi(x_1, \dots, x_n)$.

Given two automata $\mathcal{A}_{\phi_1}(\vec{x})$ and $\mathcal{A}_{\phi_2}(\vec{x})$ we can compute $\mathcal{A}_{\phi_1 \wedge \phi_2}(\vec{x})$ and $\mathcal{A}_{\phi_1 \vee \phi_2}(\vec{x})$ by the classical constructions for intersection and union of finite automata. We only have to be sure that $\vec{x}$ contains the free variables of both ϕ_1 and ϕ_2 , which can be easily be satisfied, thanks to the above exercise.

Then negation corresponds to the complement, a classical construction which is linear when the automaton is deterministic (and may be exponential otherwise). Since the automata which were constructed for atomic formulas are deterministic, computing the automaton for $a_1x_1 + \dots + a_nx_n \neq b$ is easy.

Now, it remains to handle the quantifiers. Since $\forall x.\phi$ is logically equivalent to $\neg\exists x.\neg\phi$, we only have to consider existential quantification. $\mathcal{A}_{\exists x.\phi}$ is computed from $\mathcal{A}_\phi$ by *projection*: the states, final states, initial state and transitions of $\mathcal{A}_{\exists x.\phi}$ are identical to those of $\mathcal{A}_\phi$, except that we forget the track corresponding to x in the labels of the transitions.

Example 12. Consider the formula $\exists z.x + 2y = 3z + 1$. The automaton is obtained from the automaton of figure 2.9, forgetting about z . This yields the automaton of figure 2.11. By chance, this automaton is still deterministic.

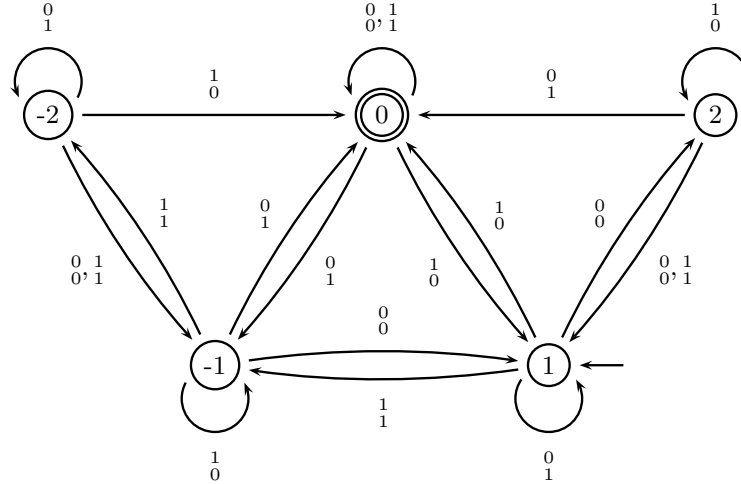


Fig. 2.11. The automaton accepting the solutions of $\exists z.x + 2y = 3z + 1$

But this is not the case in general.

Finally, by induction on the Presburger formula ϕ , we can compute an automaton which accepts the set of solutions of ϕ .

Deciding the satisfiability of a formula ϕ then reduces to decide the emptiness of $\mathcal{A}_\phi$, which can be done in linear time w.r.t. the number of states.

Consider however that $\mathcal{A}_\phi$ could have, in principle, a number of states which is a tower of exponentials whose height is the number of quantifier alternations in ϕ . This is because each quantifier alternation may require a projection followed by a complement and the projection may yield a non-deterministic automaton, requiring an exponential blow-up for the complement. Whether this multi-exponential blow-up may occur in the above construction is an open question. (Though the complexity of Presburger arithmetic is known and elementary, using other methods. See [FR79].)